#1 Method `randomEmbedding` takes a graph, bounds for a rectangle hosting the embedding (as its width and height), and the number of runs, and returns an embedding, encoded as an assignment of 2D points for each vertex.
#2 Initialization: Break down `graph` into its biconnected components (you could use connected components, to keep it simple).
#3 Repeat the algorithm `runs` times.
#4 Initialize the array of points. In the end, it will contain one point per vertex, in the same order they are stored in the graph. Alternatively, this can be a dictionary, mapping vertices to points.
#5 For each vertex in the graph…
#6 Uniformly draw a 2D point from the rectangular region of size `W*H`. Method random can return a floating point or just an integer, that's up to your implementation and requirements.
#7 Add the new point to the `embedding` array.
#8 If the assignment of vertices coordinates can't be validated (i.e. if it doesn't abide by the constraints, and at least one vertex intersects another vertex or an edge), then it discards this assignment (decrementing `i`, so that this iteration in the loop will be repeated).
#9 Compute the number of edges intersections: the method is generic, so it can support straight and curve edges, although in this case we assume the former.
#10 If this is the best result so far, store it.
#11 Return the assignment with the fewest edge intersections.

One last word of caution: we don't need to assume the graph is connected, but breaking it down into connected components before applying the heuristic can improve the final result.

In listing 16.1 we abstract two important helper methods: `validate` and `edgeIntersections`. Both can be implemented